Testing

Conventions for fast, deterministic unit tests — Arrange–Act–Assert, parameterization, and coverage floors.

Tests are JUnit 5 + AssertJ + Mockito. The rule is simple: **every new method ships with its test in the same change** — no "I'll add tests later".

Conventions

- **Arrange–Act–Assert.** Every test has three visible sections; no logic interleaved between them. Test names read as `should_<expected>_when_<condition>`.
- **Parameterized over copy-paste.** Same behaviour, different inputs → one `@ParameterizedTest` with `@CsvSource`, not N near-identical methods.
- **Unit, not integration.** Collaborators are mocked (Mockito). Testcontainers / `@SpringBootTest` wiring tests are intentionally out of scope — the suite stays fast and deterministic.
- **One reason to fail.** A test asserts one behaviour; shared setup goes in `@BeforeEach`.

```
@Test
void should_returnRealizedPnl_when_positionIsClosed() {
    // Arrange
    var position = aClosedPosition(/* entry */ 100, /* exit */ 140, /* qty */ 10);

    // Act
    var pnl = calculator.realizedPnl(position);

    // Assert
    assertThat(pnl).isEqualByComparingTo("400");
}

@ParameterizedTest
@CsvSource({ "USD, 45.71", "EUR, 49.30", "TRY, 1" })
void should_convertToTry_when_givenNativeCurrency(String currency, BigDecimal rate) {
    assertThat(converter.toTry(BigDecimal.ONE, currency)).isEqualByComparingTo(rate);
}
```

Coverage & quality gate

JaCoCo per module; the target is **≥ 80%** line coverage, with the core modules held at **≥ 85%**. Coverage is a floor for *new* code, not a number to chase by testing getters — assert behaviour.

The JaCoCo reports plus static analysis — code smells, bugs and security hotspots — are published to SonarCloud.io from CI, so quality is tracked on every change rather than only on a developer's machine.

Running

```
# backend (multi-module)
cd backend && ./mvnw -fae test            # all modules, fail-at-end
./mvnw -pl finance-portfolio test          # one module
./mvnw -pl finance-portfolio test -Dtest=PortfolioPnlCalculatorTest   # one class

# finance-common (shared lib)
cd finance-common && mvn test

# notification microservice
cd notification-backend && mvn test
```

JaCoCo writes its report to each module's `target/site/jacoco/index.html` after `mvn verify`.